## What Is Obsidian?

Obsidian is a note-taking application that stores all of your notes as plain Markdown files on your own computer. There is no proprietary format, no required account, and no dependency on a cloud service. Your notes are just text files in a folder, which means they are readable by any text editor, portable across any machine, and easy to back up with Git.

What separates Obsidian from other note-taking tools is its linking system. Every note can link to any other note, and Obsidian builds a visual graph from those connections. For math majors, this matters because mathematics is not linear. A theorem in topology depends on definitions from real analysis, which connect to ideas from set theory. Obsidian lets you build a note structure that reflects those relationships rather than forcing everything into a flat folder hierarchy or a chronological notebook.

This guide covers setting up Obsidian, writing math using MathJax, and configuring the plugins that make Obsidian practical for upper-level mathematics coursework.

---

## Table of Contents


1. Installation and First Launch
2. Understanding Vaults
3. Markdown Basics for Math Notes
4. Writing Math with MathJax
5. Linking Notes Together
6. Organizing Your Vault
7. Templates
8. Recommended Plugins
9. LaTeX Suite Plugin

10. Dataview Plugin
11. Backing Up Your Vault with Git
12. Suggested Vault Structure for Math Majors

---

### 1. Installation and First Launch

Download Obsidian from the official site:

> https://obsidian.md/download

Select the installer for your operating system and run it. Obsidian does not require a login or an internet connection to use.

On first launch, Obsidian presents three options:

- **Create new vault** -- start fresh with an empty folder
- **Open folder as vault** -- use an existing folder of files
- **Open vault from Obsidian Sync** -- for users of Obsidian's paid sync service (not required)

Select **Create new vault**. Give it a name and choose where to store it. A folder inside your Documents directory is a reasonable default. Click **Create**.

Obsidian opens to your new, empty vault.

---

### 2. Understanding Vaults

A vault is the root folder that Obsidian tracks. Everything inside it, including subfolders, is part of the vault. Obsidian stores its configuration in a hidden `.obsidian` folder at the vault root. Your actual notes are just `.md` files.

You can have multiple vaults, one per course for example, or one large vault for all of your math notes with subfolders per course. For most math majors, a single vault works better because it allows links and searches to cross course boundaries. A definition you wrote for real analysis might be relevant when you are studying functional analysis a year later.

To open your vault folder directly in your file system, right-click any note in the left sidebar and select **Show in system explorer** (macOS) or **Show in folder** (Windows). This confirms that Obsidian is working with ordinary files you can access from anywhere.

---

### 3. Markdown Basics for Math Notes

Obsidian uses Markdown for formatting. If you have used GitHub, you have already seen Markdown. Below are the elements you will use most often in math notes.

**Headings:**

```
# Heading 1
## Heading 2
### Heading 3
```

Use headings to organize a note into sections. A note on metric spaces might have headings for Definition, Examples, Theorems, and Proofs.

**Bold and italic:**

```
**bold text**
*italic text*
```

Use italic for introducing a new term and bold for emphasis on critical statements.

**Bullet lists:**

```
- First item
- Second item
  - Nested item
```

**Numbered lists:**

```
1. First step
2. Second step
3. Third step
```

**Horizontal rule (useful for separating proof from theorem):**

```
---
```

**Blockquote (useful for highlighting key statements):**

```
> A metric space is compact if and only if it is complete and
totally bounded.
```

**Code block (for Python or R snippets within a note):**

```
```python
import numpy as np
x = np.linspace(0, 1, 100)
```
```

Obsidian renders all of these in the reading view. Press `Ctrl+E` (Windows/Linux) or `Cmd+E` (macOS) to toggle between editing and reading view.

---

### 4. Writing Math with MathJax

Obsidian supports MathJax natively with no plugins required. MathJax renders LaTeX math syntax directly inside your notes.

**Inline math** is surrounded by single dollar signs and renders within a line of text:

```
The function $f: \mathbb{R} \to \mathbb{R}$ is continuous at $x_0$.
```

Renders as: The function f: R to R is continuous at x0.

**Display math** is surrounded by double dollar signs and renders on its own centered line:

```
$$
\lim_{x \to x_0} f(x) = f(x_0)
$$
```

**Block math with alignment** uses the `align` environment. This is useful for multi-step proofs:

```
$$
\begin{align}
|f(x) - f(x_0)| &= |f(x) - L + L - f(x_0)| \\
&\leq |f(x) - L| + |L - f(x_0)| \\
&< \frac{\varepsilon}{2} + \frac{\varepsilon}{2} \\
&= \varepsilon
\end{align}
$$
```

### Matrices:

```
$$
A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}
$$
```

### Common symbols used in upper-level math:

| Symbol | LaTeX |
|---|---|
| Real numbers | `\mathbb{R}` |
| Natural numbers | `\mathbb{N}` |
| Integers | `\mathbb{Z}` |
| Rational numbers | `\mathbb{Q}` |
| Complex numbers | `\mathbb{C}` |
| For all | `\forall` |
| There exists | `\exists` |
| Element of | `\in` |
| Subset | `\subset` |
| Union | `\cup` |
| Intersection | `\cap` |
| Infinity | `\infty` |
| Epsilon | `\varepsilon` |
| Delta | `\delta` |
| Partial derivative | `\partial` |
| Nabla | `\nabla` |
| Implies | `\Rightarrow` |
| If and only if | `\Leftrightarrow` |
| Norm | `\\|x\\|` |
| Inner product | `\langle x, y \rangle` |
| Sum | `\sum_{i=1}^{n}` |
| Integral | `\int_a^b` |
| Limit | `\lim_{x \to a}` |

**Tips for writing math in Obsidian:**

Switch to reading view (`Cmd+E` or `Ctrl+E`) frequently to check how your math renders. Errors in LaTeX syntax cause the block to display as raw text rather than rendered math. If a formula is not rendering, look for unmatched braces, missing backslashes, or unclosed dollar signs.

For long proofs, use the `align` environment rather than separate display blocks. This keeps the logical flow of the proof in one connected block and renders more cleanly.

---

## 5. Linking Notes Together

Linking is the feature that distinguishes Obsidian from a standard text editor. Instead of keeping separate files that have no relationship to each other, you build a network where each note connects to the ideas it depends on or relates to.

**Creating a link:**

Type `[[` and begin typing the name of another note. Obsidian shows a dropdown of matching notes. Select one or finish typing the name and press Enter.

```
The proof relies on the [[Heine-Cantor Theorem]].
```

If the note does not exist yet, Obsidian creates it when you click the link. This lets you write notes quickly and fill in definitions later.

**Linking to a specific section within a note:**

```
[[Metric Spaces#Compactness]]
```

The `#` refers to a heading inside the target note.

**Displaying different link text:**

```
[[Heine-Cantor Theorem|Heine-Cantor]]
```

The text after the pipe (`|`) is what appears in the note, while the link still points to the original note.

**Backlinks:**

Every note has a backlinks panel that shows every other note linking to it. Open it with the backlinks icon in the right sidebar. This is useful for finding every theorem that depends on a particular definition, or every proof that uses a specific lemma.

**The graph view:**

Open the graph with `Ctrl+G` or `Cmd+G`. Each note is a node and each link is an edge. As your vault grows, the graph gives you a visual map of your knowledge. Clusters of connected notes usually correspond to coherent areas of mathematics.

---

### 6. Organizing Your Vault

Obsidian supports folders, but the linking system reduces the importance of rigid folder hierarchies. A note can belong to a folder and still link freely to any other note in the vault regardless of where it is stored.

A practical approach for math majors is to organize by course at the folder level, and use links and tags to express cross-course relationships.

**Tags:**

Add tags to a note using the `#` symbol anywhere in the note body:

```
#definition #topology #compactness
```

Or add them in the note's frontmatter (a block at the top of the file):

```
---
tags: [definition, topology, compactness]
---
```

Tags let you group notes by type (definition, theorem, proof, example) across courses and folders. You can click any tag to see all notes that share it.

**Frontmatter:**

Frontmatter is a block of structured metadata at the top of a note, written in YAML and surrounded by triple dashes. Obsidian reads it and makes the fields searchable and queryable.

```
---
title: Uniform Continuity
type: definition
course: MATH 401
date: 2026-02-14
tags: [definition, analysis, continuity]
---
```

Frontmatter fields become especially useful when combined with the Dataview plugin, covered in Section 10.

---

### 7. Templates

Templates let you create a consistent structure for every new note of the same type. Instead of typing the same headings and frontmatter fields from scratch each time, you insert a template and fill in the content.

**Enable the Templates core plugin:**

1. Open **Settings** (gear icon, bottom left)
2. Go to **Core plugins**
3. Enable **Templates**
4. Go to **Settings > Templates**
5. Set the **Template folder location** to a folder called `Templates` in your vault

**Create a template:**

Create a new note inside your `Templates` folder. Here is a template for a mathematical definition:

```
---
title:
type: definition
course:
date: {{date}}
tags: [definition]
---

## Definition

## Intuition
```

```
## Examples

## Related Theorems

## Source
```

**Insert a template into a new note:**

Open a new note, then press `Ctrl+P` or `Cmd+P` to open the command palette. Type "insert template" and select your template from the list. The template content appears in the current note.

**Recommended templates for math majors:**

Definition template (as above), theorem template (with fields for statement, proof, and corollaries), and a lecture notes template (with fields for date, course, and topic). Using consistent templates makes the Dataview queries in Section 10 significantly more powerful.

---

### 8. Recommended Plugins

Obsidian separates plugins into two categories: core plugins (built into Obsidian and maintained by the Obsidian team) and community plugins (built and maintained by third-party developers).

**To install community plugins:**

1. Open **Settings**
2. Go to **Community plugins**
3. Turn off **Restricted mode** when prompted
4. Click **Browse** to open the plugin directory
5. Search for the plugin by name
6. Click **Install**, then **Enable**

The two community plugins most useful for math majors are LaTeX Suite and Dataview. Both are covered in detail in the sections below.

**Core plugins to enable:**

| Plugin | Purpose |
|---|---|
| Templates | Insert pre-built note structures |
| Backlinks | See which notes link to the current note |
| Outgoing links | See which notes the current note links to |
| Graph view | Visualize the connection network across your vault |
| Search | Full-text search across all notes |
| Quick switcher | Jump to any note by name with `Ctrl+O` or `Cmd+O` |
| Tag pane | Browse all tags used across the vault |

Enable these in **Settings > Core plugins**.

---

### 9. LaTeX Suite Plugin

**What it does:**

LaTeX Suite is a community plugin that dramatically speeds up typing mathematical notation in Obsidian. It works through snippet expansion: you type a short sequence of characters and the plugin replaces it with a longer LaTeX expression, positioning your cursor inside the result so you can keep typing immediately.

Without LaTeX Suite, writing `$\frac{d}{dx}$` takes 13 keystrokes. With a snippet, you type `//` inside a math block and the plugin expands it to `\frac{}{}` with your cursor in the numerator. This kind of acceleration matters when you are taking notes during a lecture or working through a proof quickly.

**Installation:**

Open **Settings > Community plugins > Browse**, search for **LaTeX Suite**, install it, and enable it.

**How snippets work:**

LaTeX Suite comes with a default set of snippets. Inside a math block, type the trigger sequence and the plugin expands it. The cursor lands inside the most logical position in the result.

A selection of the default snippets most useful for math majors:

| Trigger | Expands to | Notes |
|---|---|---|
| `mk` | `$ $` | Opens inline math mode |
| `dm` | `$$ $$` | Opens display math mode |
| `//` | `\frac{}{}` | Fraction |
| `sq` | `\sqrt{}` | Square root |
| `sr` | `^{2}` | Superscript 2 |
| `cb` | `^{3}` | Superscript 3 |
| `td` | `^{}` | General superscript |
| `__` | `_{}` | Subscript |
| `@a` | `\alpha` | Greek letters via @ prefix |
| `@b` | `\beta` | |
| `@e` | `\varepsilon` | |
| `@d` | `\delta` | |
| `@l` | `\lambda` | |
| `@m` | `\mu` | |
| `@p` | `\phi` | |
| `@s` | `\sigma` | |
| `@t` | `\theta` | |
| `@o` | `\omega` | |
| `⇒` | `\Rightarrow` | Implies |

| Trigger | Expands to | Notes |
|---|---|---|
| `=<` | `\Leftarrow` | |
| $\Longleftrightarrow$ | `\Leftrightarrow` | If and only if |
| $\neq$ | `\neq` | Not equal |
| $\leq$ | `\leq` | Less than or equal |
| $\geq$ | `\geq` | Greater than or equal |
| `AA` | `\forall` | For all |
| `EE` | `\exists` | There exists |
| `inn` | `\in` | Element of |
| `!in` | `\notin` | Not element of |
| `cc` | `\subset` | Subset |
| $\rightarrow$ | `\to` | Arrow |
| `!>` | `\mapsto` | Maps to |
| `RR` | `\mathbb{R}` | Real numbers |
| `NN` | `\mathbb{N}` | Natural numbers |
| `ZZ` | `\mathbb{Z}` | Integers |
| `QQ` | `\mathbb{Q}` | Rationals |
| `CC` | `\mathbb{C}` | Complex numbers |

**Writing custom snippets:**

You can add your own snippets in **Settings > LaTeX Suite**. Snippets are written in JSON format. Each snippet has a trigger, a replacement, and an optional context that restricts when the trigger activates.

Example: adding a snippet for the norm symbol:

```
{
  "trigger": "norm",
  "replacement": "\\|$0\\|",
  "options": "mA"
}
```

The `m` option means the snippet only activates inside a math block.
The `A` option means it expands automatically without pressing Tab. `$0` marks where the cursor lands after expansion.

Example: a snippet for limits:

```
{
  "trigger": "lim",
  "replacement": "\\lim_{$1 \\to $2} $0",
  "options": "mA"
}
```

`$1` and `$2` are tab stops. After the snippet expands, pressing Tab moves the cursor to each position in order.

**Tips for getting started with LaTeX Suite:**

Do not try to memorize all the default snippets at once. Start with the math mode openers (`mk` and `dm`), the fraction shortcut (`//`), and the Greek letter shortcuts (`@e` for epsilon, `@d` for delta). Add more as you notice yourself typing the same LaTeX repeatedly.

Keep the plugin's snippet reference open in a browser tab for the first few weeks until the most common ones become automatic.

---

### 10. Dataview Plugin

**What it does:**

Dataview treats your Obsidian vault as a database. It lets you write queries against your notes using the frontmatter fields you defined, and displays the results as tables, lists, or task views inside any note. For a math major with a large vault, this means you can generate dynamic indexes of every definition,

every theorem, and every proof across all your courses without maintaining those indexes by hand.

**Installation:**

Open **Settings > Community plugins > Browse**, search for **Dataview**, install it, and enable it.

**How queries work:**

Dataview queries are written inside a special code block with the language set to `dataview`. Obsidian runs the query when it renders the note and displays the results inline.

**Basic query: list all notes tagged as definitions**

```
```dataview
LIST
FROM #definition
SORT file.name ASC
```
```

This produces a linked list of every note in your vault that has the `#definition` tag, sorted alphabetically.

**Table query: index all theorems with their course and date**

```
```dataview
TABLE course, date
FROM #theorem
SORT date DESC
```
```

This produces a table with one row per theorem note, showing the course and date columns from each note's frontmatter. Sorted by most recent first.

**Query with a filter: all definitions from one course**

```
```dataview
LIST
FROM #definition
WHERE course = "MATH 401"
SORT file.name ASC
```
```

**Query: all notes modified in the last week**

```
```dataview
LIST
FROM ""
WHERE file.mtime ≥ date(today) - dur(7 days)
SORT file.mtime DESC
```
```

Useful for reviewing what you worked on recently before an exam.

**Building a course index page:**

Create a note called `MATH 401 Index` and add queries for each content type:

```
# MATH 401 Index

## Definitions

```dataview
TABLE date
FROM #definition
WHERE course = "MATH 401"
```
```

```
SORT file.name ASC
```

## Theorems

```dataview
TABLE date
FROM #theorem
WHERE course = "MATH 401"
SORT date ASC
```

## Proofs

```dataview
LIST
FROM #proof
WHERE course = "MATH 401"
```

This page updates automatically every time you add a new note with the matching tags and frontmatter. You never update the index manually.

**Tips for using Dataview effectively:**

Dataview is only as useful as your frontmatter is consistent. If some notes use `course: MATH 401` and others use `course: math401` or omit the field entirely, queries will miss those notes. Establish your frontmatter conventions early (using templates from Section 7) and stick to them. The combination of consistent templates and Dataview queries is what makes a large vault navigable.

---

### 11. Backing Up Your Vault with Git

Because your vault is just a folder of text files, Git backs it up perfectly. Refer to the Introduction to Git section of this guide for full setup instructions. Below is

the specific sequence for connecting an Obsidian vault to GitHub.

**Initialize a repository in your vault:**

```
cd path/to/your/vault
git init
git remote add origin https://github.com/yourusername/vault-repo-name.git
```

**Create a .gitignore for Obsidian:**

Some files in the `.obsidian` configuration folder change frequently and do not need to be tracked (workspace layout, cache files). Create a `.gitignore` at the vault root:

```
.obsidian/workspace.json
.obsidian/workspace-mobile.json
.obsidian/cache
.trash/
```

You can choose to commit the rest of the `.obsidian` folder (your plugin settings and configuration) so that your setup is restored if you clone the vault on a new machine.

**Initial commit and push:**

```
git add .
git commit -m "Initial vault commit"
git push -u origin main
```

**After each study session:**

```
git add .
git commit -m "Session: metric spaces, lecture 8"
git push
```

Three commands. Make it a habit at the end of every session the same way you save a file.

---

### 12. Suggested Vault Structure for Math Majors

Below is a folder structure that works well for a math major using Obsidian across multiple courses and years. This is a starting point, not a rigid requirement. Adjust it to match your courses and working style.

```
Math Notes/
|
+-- Courses/
|   +-- MATH 401 - Real Analysis/
|   +-- MATH 435 - Abstract Algebra/
|   +-- MATH 450 - Topology/
|
+-- Concepts/
|   +-- Definitions/
|   +-- Theorems/
|   +-- Proofs/
|   +-- Examples/
|
+-- References/
|   +-- Textbooks/
|   +-- Papers/
|
+-- Templates/
|   +-- Definition Template.md
|   +-- Theorem Template.md
|   +-- Lecture Notes Template.md
|   +-- Proof Template.md
|
```

```
+-- Indexes/
|   +-- MATH 401 Index.md
|   +-- MATH 435 Index.md
|   +-- Master Definition Index.md
|   +-- Master Theorem Index.md
|
+-- Scratch/
```

**How to use this structure:**

Course folders hold your lecture notes, organized by date or topic. The Concepts folders hold atomic notes (one note per definition, one per theorem) that link across courses. Index notes use Dataview queries to pull everything together automatically. The Scratch folder is for rough working notes that you may or may not develop into proper notes later.

As your vault grows over multiple semesters, the Concepts folders become the most valuable part. A note on compactness written in your first topology course becomes a reference you link to from functional analysis, measure theory, and beyond. The structure supports that kind of accumulation in a way that a folder of course-specific notebooks does not.